

人工智慧導論



Java

繼承、抽象類別和多形

本章提要

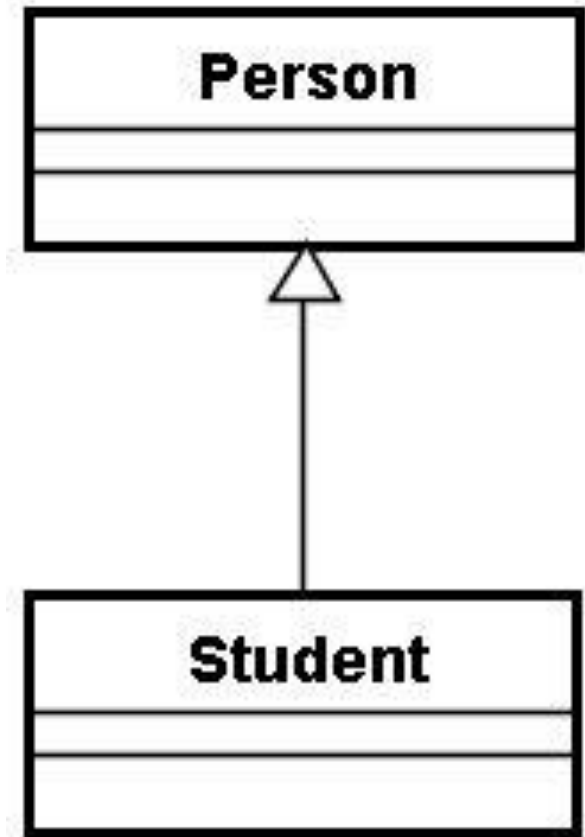
- 繼承
- 常數與抽象類別
- 多形

繼承的基礎-說明

- 「繼承」（**Inheritance**）是物件導向程式設計的重要觀念，繼承是一個類別含括現存類別的部分或全部的成員資料和成員方法，並且新增額外的成員資料或方法，或覆寫與隱藏繼承類別的方法或變數。
- 類別繼承也是在模擬真實世界，例如：學生和老師都是人，我們可以先定義**Person**類別來模擬人類，然後擴充**Person**類別建立**Student**類別來模擬學生。

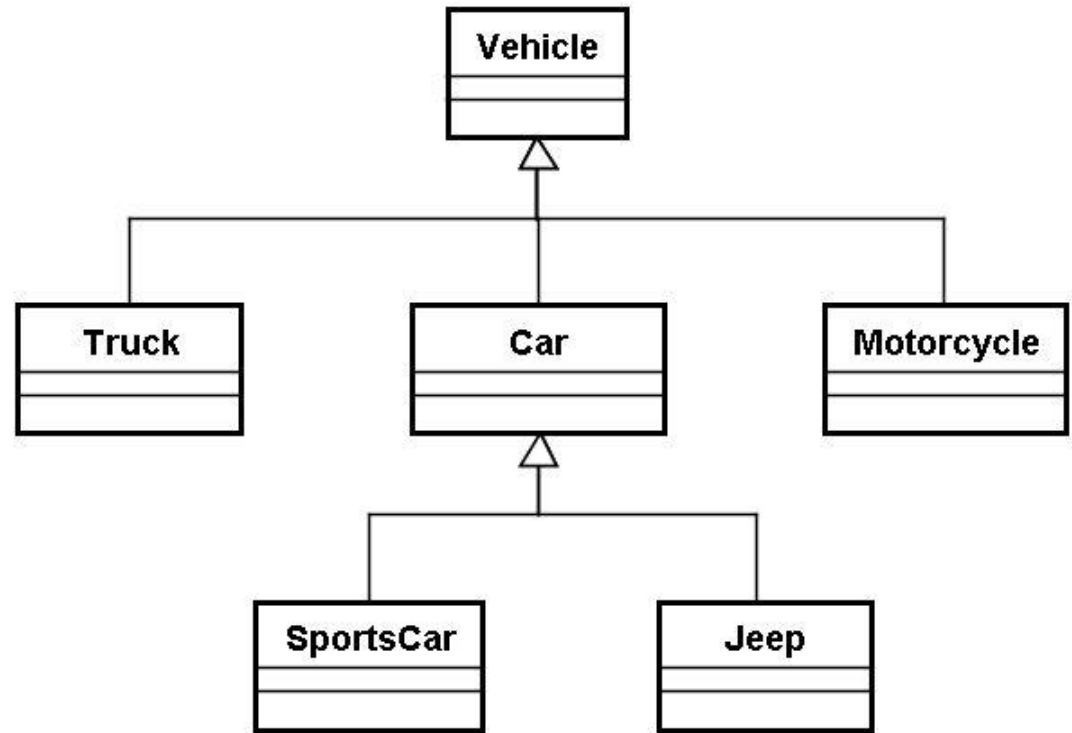
繼承的基礎-父類別與子類別

- **Person**類別是**Student**類別的「父類別」（**Superclass**）或稱為「基礎類別」（**Base Class**）；反之**Student**類別是**Person**類別的「子類別」（**Subclass**）或「延伸類別」（**Derived Class**）。UML類別圖的繼承是使用空心的箭頭線來標示兩個類別間的關係。



繼承的基礎-類別架構

- 繼承不只可以多個子類別繼承同一個父類別，還可以擁有很多層的繼承，如果將整個類別關聯性（**Relationships**）的樹狀結構都繪出來，稱為類別架構（**Class Hierarchy**），如右圖所示：



繼承的基礎-兄弟類別

- ❑ **Truck、Car和Motorcycle**類別是「兄弟類別」（**Sibling Classes**），因為擁有相同**Vehicle**父類別。當然我們可以繼續繼承類別**Car**，類別**SportsCar**和**Jeep**也是類別**Vehicle**的子類別，不過並不是直接繼承的子類別。
- ❑ **Car**類別是**SportsCar**和**Jeep**的直接父類別（**Direct Base Class**），**Vehicle**類別則是**SportsCar**和**Jeep**的間接父類別（**Indirect Base Class**）。

實作類別的繼承-父類別

- 父類別**Vehicle**定義車輛基本資料，如下所示：

```
class Vehicle {  
    private int engineNum;  
    private char color;  
    public void setNum(int num) {  
        engineNum=num; }  
    public void setColor(char color) {  
        this.color=color; }  
    public void printVehicle() { }  
}
```

7-1-2 實作類別的繼承-繼承語法

- 類別如果是繼承自存在的其他類別，其宣告語法如下所示：

```
class 子類別名稱 extends 父類別名稱 {  
    ..... // 額外的成員資料和方法  
}
```

- 語法使用**extends**關鍵字來擴充父類別的原型宣告。

實作類別的繼承-子類別

- 例如：卡車、機車和轎車等，以轎車**Car**子類別為例的類別宣告，如下所示：

```
class Car extends Vehicle {  
    private int doors;  
    public Car(int num, char color, int  
doors)  
    {  
    }  
    public void printCar() {  
    }  
}
```

實作類別的繼承-限制條件

- 雖然子類別可以繼承父類別的所有成員資料和方法，但是在存取時仍有一些限制，如下所示：
 - 子類別不能存取父類別宣告成**private**的成員資料和方法。
 - 父類別的建構子不屬於類別成員，所以在子類別並不能繼承父類別的建構子，只能呼叫父類別的建構子。

覆寫和隱藏父類別的方法-說明

- ❑ 若父類別的方法並不符合子類別的需求，我們可以在子類別宣告同名、同參數列和傳回值的方法來取代父類別的方法，稱為覆寫（**Override**），或稱為覆蓋。
- ❑ 不過，物件的實例方法並不能取代類別方法（宣告成**static**的方法），如果父類別擁有類別方法，子類別需要宣告同樣的類別方法來取代它，稱為「隱藏」（**Hide**）。

覆寫和隱藏父類別的方法-差異

□ 覆寫和隱藏的差異，如下表所示：

	父類別的實例方法	父類別的類別方法
子類別的實例方法	覆寫	編譯錯誤
子類別的類別方法	編譯錯誤	隱藏

覆寫和隱藏父類別的方法-範例(父類別)

- 父類別**Vehicle**擁有類別方法**printName()**和成員方法**printVehicle()**需要被隱藏和覆寫，如下所示：

```
class Vehicle {  
    public static void printName() { ... }  
    .....  
    public void printVehicle() { ... }  
}
```

覆寫和隱藏父類別的方法-範例(子類別)

- 子類別**Car**是繼承自父類別**Vehicle**，如下所示：

```
class Car extends Vehicle {  
    public static void printName() { ... }  
  
    .....  
  
    public void printVehicle() { ... }  
}
```

隱藏父類別的成員資料-說明

- **Java**語言不只可以在子類別覆寫和隱藏父類別的成員和類別方法，還可以隱藏父類別成員資料的變數，只需變數名稱相同，就算資料型態不同、存取敘述修飾子**public**或**private**不同，也一樣可以隱藏。

隱藏父類別的成員資料-父類別

- 例如：父類別**Vehicle**宣告成**public**的成員變數**color**是**char**字元型態，如下所示：

```
class Vehicle {  
    public char color;  
  
    .....  
}
```


隱藏父類別的成員資料-子類別

- 子類別**Car**是繼承自父類別**Vehicle**，如下所示：

```
class Car extends Vehicle {  
    private int color;  
  
    .....  
}
```

使用父類別的建構子-說明

- 在Java語言的子類別並不能繼承父類別的建構子，子類別只能使用**super**關鍵字呼叫父類別的建構子，同理，在子類別覆寫的方法和隱藏的成員變數，也都可以使用**super**關鍵字來呼叫和存取。
 -

使用父類別的建構子-父類別

- 例如：父類別**Vehicle**擁有建構子，如下所示：

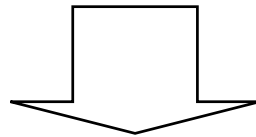
```
class Vehicle {  
    public static int count;  
    private int serial_num;  
    public int engineNum;  
    public char color;  
    public Vehicle(int num, char color, int  
no) {    }  
    public void printVehicle() {    }  
}
```

使用父類別的建構子-子類別1

- 子類別**Car**是繼承自父類別**Vehicle**，如下所示：

```
class Car extends Vehicle {  
    private char color;  
    private int engineNum;  
    private int doors;  
    public Car(int num, char color, int no,  
int doors) {  
        super(num, color, no);  
        .....
```

```
}
```



使用父類別的建構子-子類別2

```
public void printVehicle() {  
    .....  
    super.printVehicle();  
    .....  
    System.out.println("車輛色彩(父): " +  
super.color);  
    .....  
    System.out.println(super.engineNum);  
    .....  
}  
}
```

常數類別與方法-使用原因

- **Java**類別宣告成**final**表示類別不能被繼承，方法宣告成**final**就表示方法不可以覆寫，至於為什麼使用**final**修飾子？其理由如下所示：
 - 保密原因：基於保密理由，可以將一些類別宣告成**final**，以防止子類別的繼承，存取或覆寫原類別的操作。
 - 設計原因：基於物件導向設計的需求，可以將某些類別宣告成**final**，以避免子類別的繼承。

常數類別與方法-常數類別

- 例如：繼承父類別Person的Student類別，如下所示：

```
final class Student extends Person {  
    .....  
}
```

常數類別與方法-常數方法

- 在**Person**類別的某些方法則宣告成**final**，如下所示：

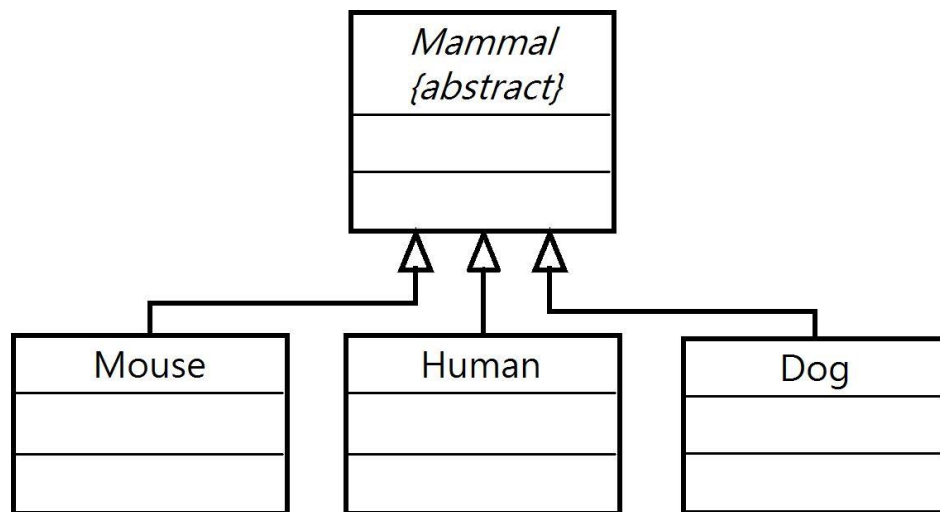
```
class Person {  
    .....  
    final char getName() { return name; }  
    final int  getAge()  { return age; }  
    final void setName(char c) { name = c;  
    }  
    final void setAge(int age) { this.age =  
    age; }  
}
```


抽象類別與方法-抽象類別的基礎

- ❑ 「抽象類別」（**Abstract Class**）是一種不能完全代表物件的類別，換句話說，它並不能建立物件，其主要目的是作為類別繼承的父類別，用來定義一些子類別的共同部分。
- ❑ 抽象類別並不能建立物件，只能被繼承用來建立子類別。例如：並沒有任何動物叫哺乳類，所以並不會建立模擬哺乳類的物件，這個類別只是描述哺乳類動物的共同特徵，以便其他屬於哺乳類的動物繼承此物件。

抽象類別與方法-抽象類別的範例

- 類別架構的父類別是哺乳類（**Mammal**），**Mouse**、**Human**和**Dog**類別是繼承自**Mammal**類別，因為老鼠、人和狗都屬於哺乳類動物，可以使用**Mouse**、**Human**和**Dog**類別建立模擬老鼠、人和狗的物件。



抽象類別與方法-抽象類別的宣告 (說明)

- ▣ **Java**類別宣告成**abstract**表示它是一個抽象類別，抽象類別不能用來建立物件，只能繼承抽象類別來宣告子類別。在抽象類別同時可以使用**abstract**宣告方法為抽象方法，表示方法只有原型宣告，實作的程式碼是在子類別建立，而且繼承的子類別一定需要實作這些抽象方法。
- ▣ 抽象類別是子類別的原型宣告，抽象方法是宣告子類別使用介面的方法，如果類別擁有抽象方法，表示類別一定是抽象類別。

抽象類別與方法-抽象類別的宣告 (範例)

- 例如：抽象類別**Shape**，如下所示：

```
abstract class Shape {  
    public double x;  
    public double y;  
    abstract void area();  
}
```

抽象類別與方法-繼承抽象類別

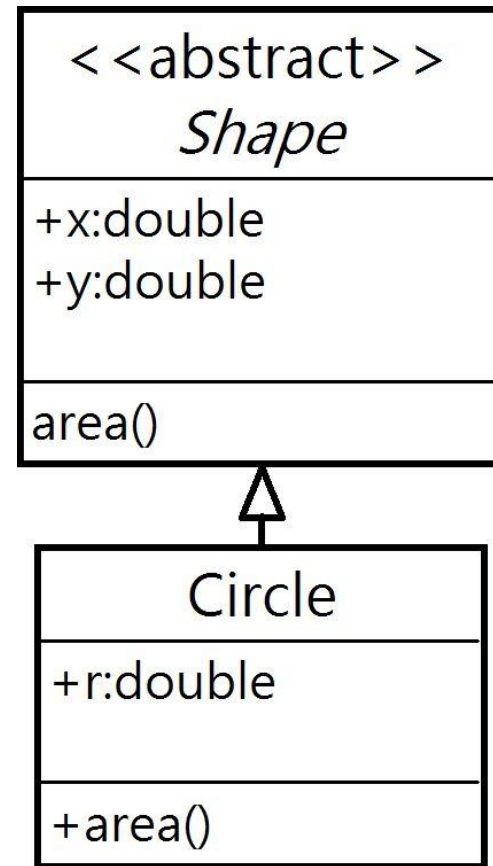
- 接著宣告**Circle**類別繼承**Shape**類別，如下所示：

```
class Circle extends Shape {  
    public double r;  
    public Circle(double x, double y, double  
r) { ... }  
    public void area()  
    { System.out.println("圓面積: " +  
3.1416*r*r); }  
}
```

- 子類別**Circle**定義圓形，除了圓心座標外，新增成員變數半徑**r**，並且實作**area()**方法計算圓面積

抽象類別與方法-UML類別圖

- 抽象類別繼承的UML類別圖，如右圖所示：



抽象類別與方法-宣告物件變數

- **Shape**抽象類別雖然不能建立物件，但是可以宣告成參考子類別實例的物件變數，如下所示：

```
Shape c2 = new Circle(10.0, 10.0, 7.0);
```

- 上述物件變數**c2**是使用抽象類別來宣告，其參考物件是**Circle**類別的實例，因為父類別的物件變數可以參考子類別的物件，以此例**Circle**也是一種**Shape**，所以**Shape**可以參考**Circle**物件。

抽象類別與方法-

判斷物件變數參考哪一種物件

- 因為**Shape**類別宣告的物件變數可以參考子類別的實例，所以**Java**程式可以使用**instanceof**運算子判斷物件變數參考哪一種物件？如下所示：

```
if ( c2 instanceof Circle) { ..... }
```

- 上述的**if**條件檢查物件變數**c2**是否是參考**Circle**物件，如果**true**就執行程式區塊的程式碼。

抽象類別與方法-呼叫或存取子類別新增的實例變數和方法

- ▣ **Shape**類別的物件變數**c2**雖然可以參考**Circle**物件，但是並不能呼叫或存取子類別新增的實例變數和方法，如果需要呼叫或存取，請先型態轉換成**Circle**類別的物件變數，如下所示：

Circle c;

c = (Circle) c2;

- ▣ 物件變數**c**是類別**Circle**宣告的變數，在經過型態轉換後，就可以存取子類別新增的實例變數和方法，例如：**c.r**。

多形-說明

- 多形是物件導向程式設計的重要觀念，可以讓應用程式更容易擴充，因為不需要針對不同資料型態分別建立類別，而是繼承一個基礎類別來建立同名方法，如此就可以處理不同資料型態，如果有新的資料型態，也只需新增繼承的子類別即可。
 -

多形-抽象類別宣告

- 物件導向技術的多形是一種複雜觀念，筆者準備使用範例來說明多形。例如：使用上一節的 **Shape** 抽象類別作為基礎類別，如下所示：

```
abstract class Shape {  
    public double x;  
    public double y;  
    abstract void area();  
}
```

多形-繼承抽象類別

```
class Circle extends Shape {  
    .....  
    public void area() { ... }  
}  
class Rectangle extends Shape {  
    .....  
    public void area() { ... }  
}  
class Triangle extends Shape {  
    .....  
    public void area() { ... }  
}
```

多形-同名異式的多形方法

- 現在我們可以宣告**Shape**類別的物件變數**s**，如下所示：

Shape s;

- 上述物件變數**s**能夠參考**Circle**、**Rectangle**和**Triangle**物件（請參閱第7-2-2節），以便呼叫各物件的方法**area()**，如下所示：

s.area();

- 上述呼叫會依照物件變數**s**參考的物件，而呼叫正確的實例方法。例如：如果**s**參考的是**Rectangle**物件，就會呼叫**Rectangle**物件的實例方法**area()**。

多形-執行過程圖例

- 物件導向程式的物件呼叫一個實例方法就是送一個訊息給物件，告訴物件需要執行什麼方法，現在 **s.area()** 是將這個訊息送到 **s** 物件變數所參考的物件，執行該物件的實例方法，如右圖所示：

